



Apex Architecture

How a self-hosted Formula 1 live timing, replay and telemetry application is put together — every component, every connection, and the reasoning behind each one.

DOCUMENT

Technical report

PROJECT

Apex

OWNER

CloudSheep

DATE

08 September 2026

— Contents

1	Overview	What Apex is, and the five things it must get right
2	System context	Which component talks to which, and in which direction
3	The path of one update	From the wire to the screen
4	The delta merge	The single most important algorithm here
5	Getting live data	Three sources behind one interface
6	Talking to the browser	Snapshot, deltas, resumption and delay
7	Storage	Why the archive is the database
8	Analysis and telemetry	Derived once, read many times
9	The frontend	Why the live state is not in a store
10	Deployment	Two containers, one volume
11	Decisions	What was chosen, and what would change it

FIGURES

1	System context — every component and the direction of every arrow
2	One update, from the wire to the screen
3	The delta merge — the single most important algorithm in the project
4	Three ways to get live data, behind one interface
5	Snapshot, deltas, and what a reconnect actually costs
6	What is on disk, and why there is no database
7	Deployment topology, with the relay fallback

01 Overview

Apex is a self-hosted Formula 1 live timing, replay, telemetry and analysis application. It follows a session as it happens, replays any session back to 2018 from a local archive, and derives per-lap analysis without a human watching it.

Everything in this report follows from five constraints that were fixed before any code was written.

CONSTRAINT	WHAT IT FORCES
The feed sends deltas	State is never transmitted whole. Both server and browser must reconstruct it by merging, using <i>identical</i> rules, or they diverge silently.
F1 runs ~24 weekends a year	The live path cannot be a separate code path. If it were, the replay path would be the tested one and live would break on race day.
Silent staleness is the worst failure	A frozen dashboard that still looks live is undetectable by the user. Every connection state is stated in words.
One person maintains this	Every moving part must earn its place. A container that has to be running, backed up and version-matched is a real cost.
It is unofficial	No F1 branding anywhere; the legal notice appears on every page.

2	0	0	90
CONTAINERS	DATABASES	BACKEND PACKAGES	BACKEND TESTS

The backend has no NuGet dependencies at all — only the .NET base class library. That is not minimalism for its own sake: the SignalR protocol, the archive format and the merge rules are all things this project had to implement exactly, and a library that almost does it would have been harder to correct than writing it.

02 System context

Four external services, two containers, one volume. Nothing else.

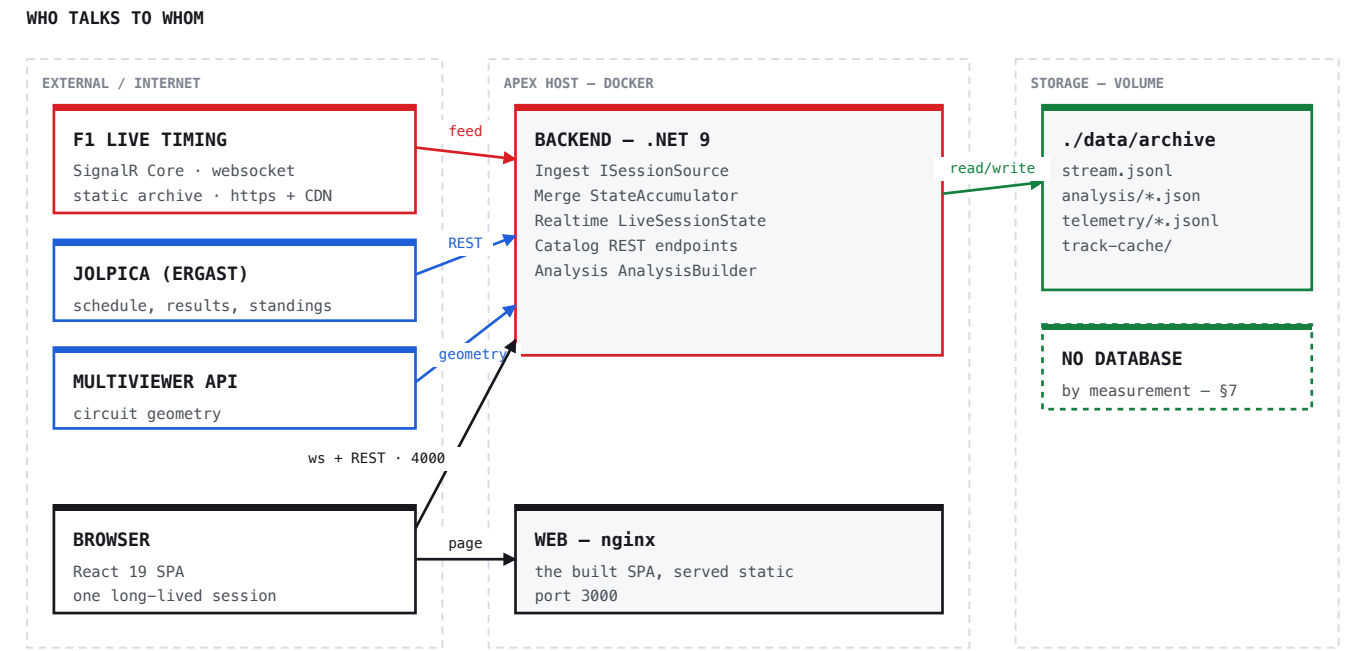


FIGURE 1 SYSTEM CONTEXT – EVERY COMPONENT AND THE DIRECTION OF EVERY ARROW

THE EXTERNAL DEPENDENCIES

SERVICE	PROVIDES	IF IT IS UNAVAILABLE
F1 live timing	The session feed and the session archive	Live stops; replay of already-downloaded sessions is unaffected.
Jolpica (<i>Ergast successor</i>)	Schedule, results, standings	Those pages degrade; timing and telemetry are untouched.
MultiViewer	Circuit geometry for the track map	Cached on disk after the first fetch, so only a new circuit is affected.

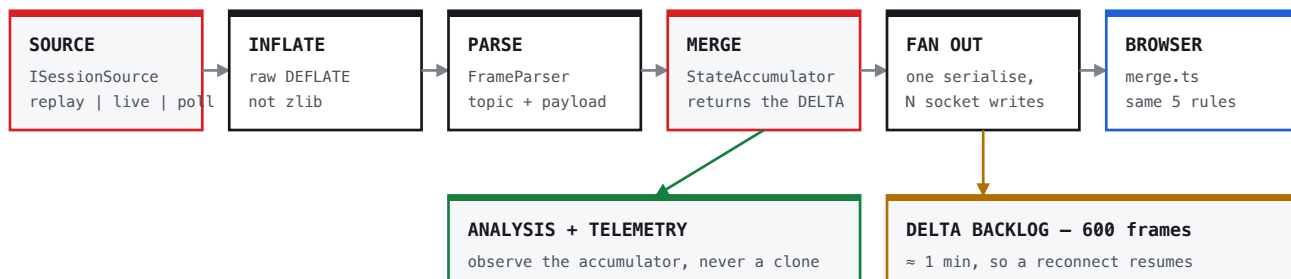
DESIGN RULE

No external service sits in the live path except F1 itself. A standings API being slow must never be able to delay a position update.

03 The path of one update

This is the hot path. Roughly ten deltas a second arrive during a session, each patching a deeply nested object, and every decision below exists to keep that cheap.

THE PATH OF ONE UPDATE



The browser runs the SAME merge algorithm the server used to produce the delta.
Both are tested against one shared fixture set, so they cannot drift apart.

FIGURE 2 ONE UPDATE, FROM THE WIRE TO THE SCREEN

TWO DETAILS THAT ARE EASY TO GET WRONG

Raw DEFLATE, not zlib

Compressed topics arrive as raw DEFLATE with no zlib header. Using `ZLibStream` fails on the first byte; `DeflateStream` is correct. The symptom is a total failure rather than a subtle one, which is fortunate.

Serialise once, write many

A delta is serialised a single time and the same byte array is handed to every connected client. This is what keeps fan-out nearly free: the number of viewers affects only the number of socket writes, never the amount of JSON work.

COST AVOIDED

The naive design rebuilds and sends the whole state on every delta. At a 1–2 MB state and ten deltas a second that is **10–20 MB/s of pure waste** — the single largest cost in the obvious implementation, and the reason the snapshot is cached and rebuilt only on demand.

BACKPRESSURE

Each client has a bounded queue. When it fills, the client is dropped and must reconnect — it is never accommodated, because one stalled browser tab must not be able to back up the shared reader for everyone else.

The queue refuses new frames rather than evicting old ones. Dropping the oldest delta would leave the client receiving patches for a state it never received: still rendering, quietly wrong. Refusing lets the socket close and forces a clean resync.

04 The delta merge

If one rule in this section is wrong, every number on the screen is wrong in a way that looks plausible. It is the most carefully tested code in the project.

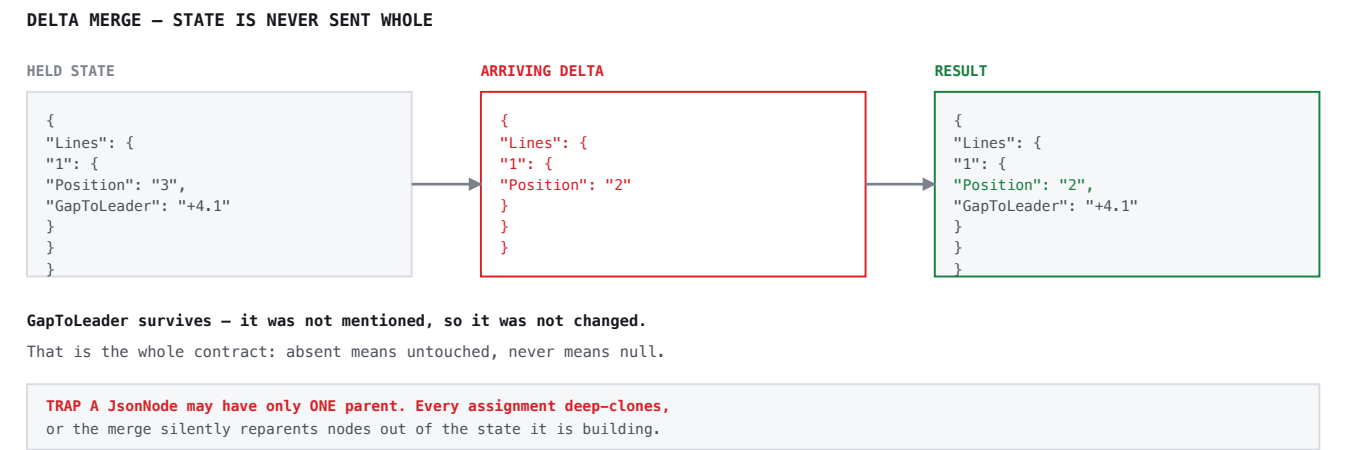


FIGURE 3 THE DELTA MERGE – THE SINGLE MOST IMPORTANT ALGORITHM IN THE PROJECT

THE FIVE RULES

CASE	BEHAVIOUR
Key absent from the delta	Left completely untouched. <u>Absent never means null.</u>
Both sides are objects	Merge recursively.
Scalar, or type change	Replace outright.
Array-like object (<i>numeric keys</i>)	Patch by index — the feed sends <code>{"2": {...}}</code> to mean "the third element".
Explicit <code>null</code>	Remove the key.

THE TRAP THAT COST THE MOST TIME

A `JsonNode` may have **only one parent**. Assigning a node that already lives elsewhere in the tree silently reparents it, corrupting the state it is being merged into. Every assignment therefore deep-clones. This is invisible in small tests and catastrophic in a real session.

PARITY BETWEEN THE TWO IMPLEMENTATIONS

The server merges in C#; the browser merges the same deltas in TypeScript. Two implementations of one algorithm is exactly the situation where drift is inevitable — so both are tested against the same fixture files on disk. A change to one that is not made to the other fails the suite.

WHAT THE ACCUMULATOR ADDS ON TOP

Not every topic merges. Three are replaced wholesale — `Position`, `CarData` and `SessionInfo`, because each publish is a complete new value and merging them would accumulate stale entries. Two are append-only — `RaceControlMessages` and `TeamRadio`, which are logs, not state.

05 Getting live data

Live data is the only genuinely fragile part of this system, because it is the only part that depends on someone else's server being willing to talk to yours. So there are three ways in, they are interchangeable, and the backend moves between them without being told to.

THREE INGEST SOURCES, ONE INTERFACE

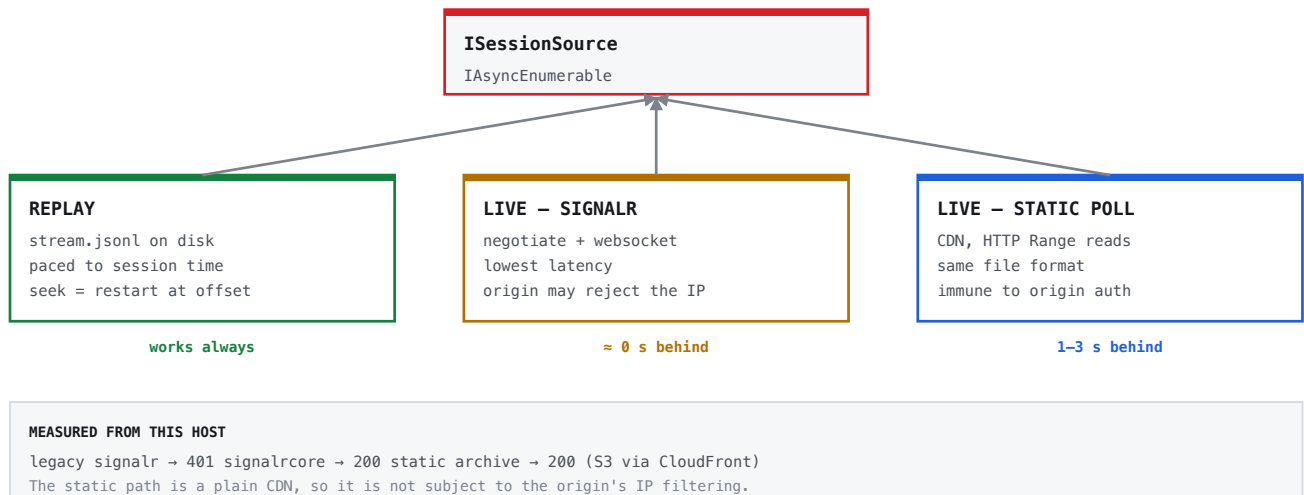


FIGURE 4 THREE WAYS TO GET LIVE DATA, BEHIND ONE INTERFACE

WHAT WAS MEASURED

From a development machine, on the same day:

```
legacy signalr/negotiate → HTTP 401
signalcore/negotiate → HTTP 200
static/SessionInfo.json → HTTP 200
                        server: AmazonS3 · via: CloudFront
```

This matters more than it first appears. The static archive is served from S3 behind CloudFront — an ordinary content delivery network — while the SignalR origin is the component that rejects requests. A deployment blocked at the origin is therefore not necessarily blocked at the CDN, and the polling source exists to exploit exactly that.

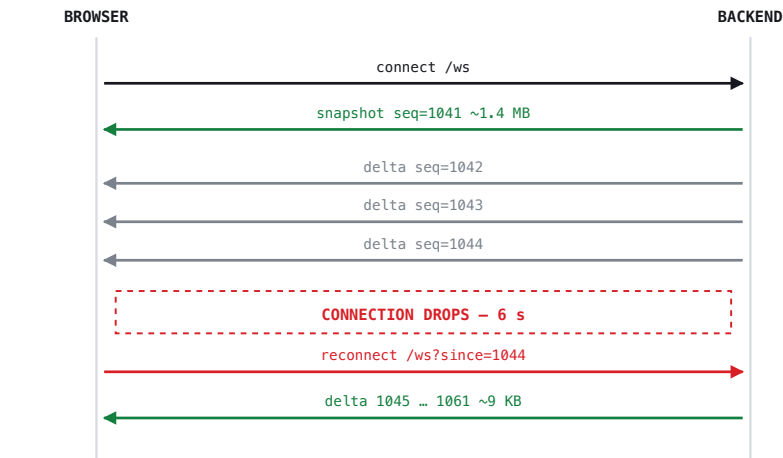
The static files support HTTP `Range` requests, confirmed with a `206 Partial Content` response, so a poller re-reads only the bytes appended since its last check — a few kilobytes a second rather than a 5.6 MB file. The format is identical to the archive the replay source already parses, including the UTF-8 byte order mark, so no new parser is involved.

WHY ONE INTERFACE MATTERS

`ISessionSource` yields `TopicUpdate` values and nothing downstream knows which implementation produced them. Replay is therefore not a testing convenience — it is the same code path live uses, exercised every single day rather than on 24 weekends a year.

06 Talking to the browser

REALTIME PROTOCOL AND RECONNECT



WHY THE BACKLOG IS BOUNDED AT 600 FRAMES

≈ one minute of live rate. Beyond it the client gets a full snapshot instead – a gap in a delta stream is unrecoverable, and guessing would corrupt the client silently.

FIGURE 5 SNAPSHOT, DELTAS, AND WHAT A RECONNECT ACTUALLY COSTS

THE CONTRACT

A connecting client receives a **snapshot** carrying the full state and a sequence number, then every subsequent **delta** in order. The client merges deltas into the snapshot with the same five rules the server used.

A snapshot **replaces** state; it never merges into it. This is not a detail: on a session change the server sends a deliberately *empty* snapshot, and a client that merged it would keep the previous session's drivers — which once rendered a 26-car field with two cars sharing first place.

RECONNECTION

The client reconnects with exponential backoff and $\pm 20\%$ jitter, so a room full of tabs does not reconnect in lockstep. It sends the last sequence it applied; the server replies with the missing deltas when its backlog reaches that far, and with a full snapshot when it does not.

CONCURRENCY

Working out what a client missed and registering it for future deltas happen **under one lock**. A delta broadcast between those two steps would be lost, and a lost delta is unrecoverable: client state is delta-accumulated, so it would stay silently wrong for the rest of the session.

HEARTBEAT

A socket that is open but dead is common on mobile networks and behind corporate proxies. Without an explicit liveness check the dashboard freezes on stale data while still looking live. Silence for longer than two heartbeat intervals closes the socket and triggers a reconnect.

BROADCAST DELAY

Television is typically 5–60 seconds behind the timing feed, so an undelayed dashboard spoils an overtake before it is shown. Messages are held in a buffer and released against the server's clock corrected for skew — timing against arrival would drift with every network hiccup, and the drift only ever accumulates.

Lowering the delay drains at a bounded multiple of real time, so going from 60 seconds to zero animates rather than teleporting the field. Snapshots bypass the buffer entirely: a snapshot is not an event to be shown later, it is the state the queued deltas patch.

07 Storage

STORAGE – THE ARCHIVE IS THE DATABASE

data/archive/		
2026/		
italian-grand-prix/		
race/		
stream.jsonl	104 MB	source of truth, append-only
analysis/		
meta.json	236 B	
laps.json	182 KB	
stints.json	6 KB	
telemetry/*.jsonl	10 MB	21 drivers, per lap
track-cache/		circuit geometry

EVERYTHING UNDER analysis/ IS DERIVED

It can be deleted and rebuilt from stream.jsonl. That is what makes a database optional rather than load-bearing here.

11 ms read one driver's whole race telemetry	0 pkgs NuGet dependencies in the backend	58 MB resident memory at steady state	0.52 % CPU one replayed session
---	---	--	--

FIGURE 6 WHAT IS ON DISK, AND WHY THERE IS NO DATABASE

THE QUESTION, ASKED PROPERLY

InfluxDB and MongoDB were both offered as free choices. Rather than reason about which shape of database fits telemetry in the abstract, the actual access patterns were measured.

ARTEFACT	SIZE	ACCESS PATTERN	COST
stream.jsonl	104 MB	Streamed, never held whole	–
analysis/*.json	~190 KB	Written once, read whole	< 5 ms
telemetry/*.jsonl	10 MB	One driver, one session	11 ms

Eleven milliseconds to read a driver's entire race telemetry from disk. A database would replace that with a network round-trip to a container that must be running, backed up and version-matched — and win nothing.

THE TRIGGER, WRITTEN DOWN IN ADVANCE

Both answers are about **per-session** access. The moment a screen asks a question spanning sessions — *"every lap over 330 km/h this season"*, *"Monza sector 2 across three years"* — the file layout stops working, because answering means opening every file in the archive. At that point InfluxDB goes in for telemetry and the analysis documents get indexed. The decision is recorded as **D-011**, and the migration is designed as **Phase K**, so it is planned rather than improvised.

TWO RULES THAT HOLD REGARDLESS

- **The archive stays authoritative.** Any store added later is a derived index that can be dropped and rebuilt, so a corrupt database is an inconvenience rather than data loss.

- **No database in the ingest path.** Adding a network write per delta would trade the one thing this application cannot afford to lose — latency during a live session — for a convenience it does not need.

08 Analysis and telemetry

Analysis is not produced by replaying a session in real time. A finished race is read from `stream.jsonl` at full speed and the results are written to disk; a two-hour race is processed in about a second and a half.

WHAT IS DERIVED

ARTEFACT	CONTENTS
Lap times	Every lap for every driver, with sector times and track status.
Stints	Compound, age, and where each stop fell.
Position history	Per-lap classification, for the progression chart.
Telemetry	Speed, throttle, brake, gear, RPM and DRS, per lap, per driver.

TWO BUGS WORTH RECORDING

Telemetry was capped at 2,000 samples per lap after lap 1 of one race recorded 12,922 — the sampling rate is not constant, and an uncapped buffer grows without bound.

Lap charts reject outliers around the median. A 1,958-second lap under a red flag destroyed the scale of an entire chart, and the track status had already cleared by the time the lap completed, so the flag could not be used to filter it.

THE 2026 FEED CHANGE

DRS was removed from the regulations and from the feed: `CarData` channel 45 is simply absent. The replacement aids are not published at all, so nothing can be shown for them; what 2026 added instead is an overtake counter.

The era is detected from the payload, not from the season — if channel 45 is present it is a DRS era, otherwise the overtake series is used. A mid-season feed change therefore degrades gracefully instead of breaking.

09 The frontend

React 19 and Vite, TypeScript in strict mode with `noUncheckedIndexedAccess` and `exactOptionalPropertyTypes`. Routing is explicit: `/live` and `/replay` are separate destinations, never tabs over one shared session, because a user cannot tell a replay from live by looking at the numbers.

THE ONE DOCUMENTED EXCEPTION

Live session state lives outside React entirely. Ten deltas a second dispatched through a normal store would re-render the tree and drop frames. Instead deltas are merged into a module-level object, and subscribers are notified at most once per animation frame through `useSyncExternalStore`. The projection from raw state to view models runs once per frame, not once per delta.

WHAT THE FEED DOES NOT KEEP

The feed publishes the **present**: a driver's current last-lap time, the track status right now. It has no history at all. Anything drawn over time — a pace chart, a timeline of safety car periods — is accumulated client-side by watching the stream. This is deliberately lossy: joining a session halfway means the history starts halfway, which is stated in the interface rather than disguised. The complete version of the same data is what the backend writes to disk.

RESPONSIVE LAYOUT

Three layouts at 768 px and 1280 px. The breakpoints are subscribed to through `matchMedia` rather than a resize listener — a resize handler fires on every pixel of a drag, while a media query fires only when the answer changes. It also means the JavaScript and the stylesheet can never disagree about which layout is active.

CHARTS ARE SVG, DELIBERATELY

PDF export is a browser print. An SVG chart prints as vectors; a canvas chart prints as a screenshot. Lines also break at missing data rather than bridging it — joining across a lap a driver did not set would draw a segment through data that does not exist, and it would look exactly like a real lap.

10 Deployment

DEPLOYMENT — AND THE RELAY FALLBACK

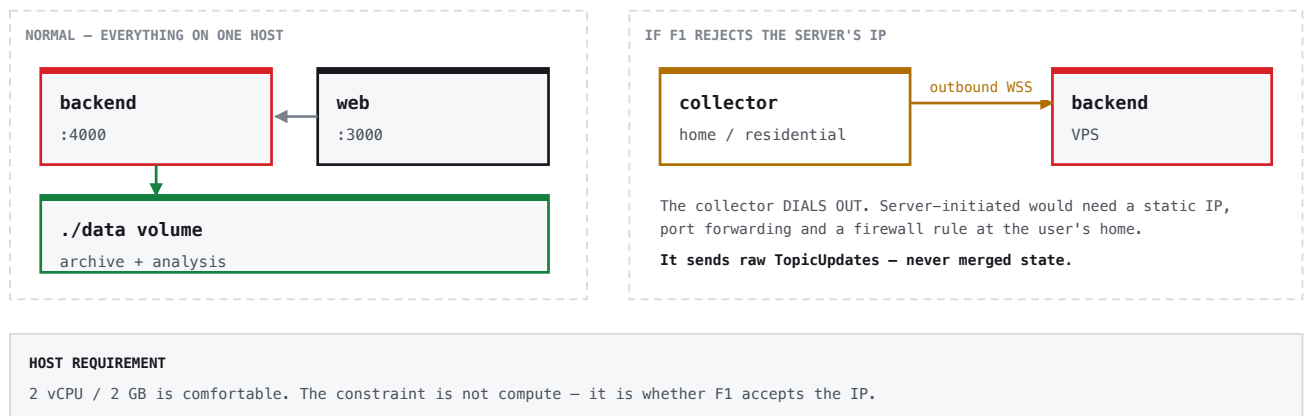


FIGURE 7 DEPLOYMENT TOPOLOGY, WITH THE RELAY FALLBACK

The host needs **only Docker**. Even the .NET SDK runs in a container for development, so no toolchain is installed on the machine.

RESOURCE USE, MEASURED

0.52% CPU, ONE SESSION	58 MB RESIDENT	80–122 MB PER SESSION	2 / 2 RECOMMENDED VCPU / GB
----------------------------------	--------------------------	---------------------------------	---------------------------------------

The real constraint is **not compute**. It is whether F1 accepts the server's IP address — which is why §5 exists, and why the relay above is designed rather than improvised.

ONE DETAIL THAT COST AN AFTERNOON

The container health check must use `127.0.0.1`, not `localhost`. `localhost` resolves to the IPv6 loopback, and Kestrel binds IPv4 — so the check fails while the service is perfectly healthy.

THE RELAY, IF IT IS NEEDED

A collector process runs where F1 accepts connections — a home connection is the most reliable, being a residential address — and **dials outward** to the server. That direction is the whole design: a server-initiated connection would require a static IP, port forwarding and a firewall rule at the user's home, and would break the first time their provider rotated the address.

It forwards raw topic updates, never merged state. Sending merged state would mean shipping 1–2 MB per update instead of a few kilobytes, and would put a second copy of the merge algorithm in production where it could drift from the first.

11 Decisions

Every departure from the original design is recorded with its reasoning in `DECISIONS.md`. The ones that shape the architecture:

ID	DECISION	REASONING
D-001	.NET 9 rather than Python	Throughput was never the constraint; the deciding factor was a typed model and channel-based fan-out mapping cleanly onto the bounded-queue design.
D-003	Vite SPA rather than Next.js	There is nothing to server-render. The application is one long-lived session driven by a socket.
D-004	Live state outside React	Ten deltas a second against a deep object. The one documented exception to the state rules.
D-009	One backend; Redis deferred	Redis existed to bridge a split between ingest and fan-out. Without the split there is nothing to bridge. The seam is preserved — replacing it touches one file.
D-010	Bounded channels use <code>Wait</code>	<code>DropWrite</code> discards silently <i>and returns success</i> , which is indistinguishable from delivery. Caught by a test, not by observation.
D-012	Three live sources	Failover is judged on <i>data delivered</i> , not on connection — the observed failure is a handshake that completes and then goes silent.
D-011	No database yet	Measured, not assumed: 11 ms to read a race of telemetry. The trigger for revisiting is written down rather than left to judgement.

VERIFICATION

The strongest evidence that the ingest layer is correct is a single assertion: replaying the 2024 Italian Grand Prix through the accumulator produces a classification that matches the official top ten exactly. Everything upstream of that — inflate, parse, merge, accumulate — has to be right for it to hold.

STATUS

110 backend tests and 55 in the browser suite, with a typechecked and linted frontend. All three ingest sources are built and each is verified against something real: the poller against F1's live archive, the relay end to end over websockets. Live SignalR subscription remains the one open item — the handshake completes and the initial state arrives in a Node probe, while the identical byte-for-byte payload is rejected in C#. It is no longer a blocker, which is why §5 is ordered the way it is.